



CYBERSTER PURPLE TEAM INTERNSHIP

Batch 1 — Month 2: Network Security & Protocol Analysis

WEEK 5 TECHNICAL REPORT

Intern Name	Shakera Karolia
Roll Number	CSI-B1-516
Program	Cyberster Internship Program — Batch 1
Team	Purple Team
Instructor	Muhammad Saad
Report Date	19 May 2026
Week	Week 5 of 12
Module	Month 2, Week 5: Network Detection & Defense Engineering

CONFIDENTIAL - For Cyberster Internal Review Only

Shakera Karolia | CSI-B1-516 | Page 1

Executive Summary

This technical report documents the engineering design, deployment, and validation methodologies executed during Week 5 of the Cyberster Purple Team Internship Program. The operational objective is to analyze the structural traffic handling differences between passive detection and inline prevention appliances.

Using standard Linux kernel architectures and matching Suricata signature formatting syntax, we designed custom rules targeting baseline protocol identifiers, high-velocity Layer 4 Availability threats (TCP SYN Floods), and unauthorized Layer 2 Man-in-the-Middle (MitM) positioning vectors. This validation guarantees that enterprise defensive systems capture real-time telemetry while ensuring business service reliability.

Task 01: Intrusion Handling Engine Architecture

1.1 Appliance Placement and Traffic Traversal Logic

- **Intrusion Detection System (IDS Mode):** Configured as an out-of-band monitoring node. The sensor is attached via a physical Network Tap or a Switch Port Analyzer (SPAN) / mirror port configuration. Traffic streams run undisturbed across production lines while the engine inspects a cloned duplicate of the packets.
- **Intrusion Prevention System (IPS Mode):** Configured inline within the active communications stream. Data packets must physically pass through the appliance network inspection engine before reaching their targeted application interfaces.

1.2 Operational Performance Matrix

Evaluation Dimension	Passive Monitoring (IDS Mode)	Inline Prevention (IPS Mode)
Data Flow Alignment	Out-of-band (Copied Stream)	In-line (Active Transport Path)
Core Defensive Capability	Read-Only Alerting & Telemetry	Real-Time Packet Dropping / Mitigation
System Latency Overhead	\$0\%\$ Impact on Network Throughput	Variable processing overhead per packet
Hardware Failure Mode	Fail-Open (Traffic flows if engine stops)	Fail-Closed / Custom (Can drop connectivity)

1.3 Technical Action Constraint Analysis: Why Detection Cannot Mitigate

An IDS cannot natively block an active attack because it evaluates traffic on a mirrored copy of the packet stream. By the time the parsing matrix decodes the copied headers and matches the signature fields to flag an active exploit, the original malicious packet has already traversed the network switch fabric and reached its destination endpoint unimpeded.

Task 02: Laboratory Simulation & Forensic Logging

2.1 Passive Baseline Traffic Ingestion Log (Day 1)

Using a Localized Virtualization Model to isolate processing loops from hypervisor driver conflicts, baseline out-of-band capture states were validated via the native loopback framework (lo).

- **Active Monitoring Execution:** `sudo tcpdump -i lo -w lo_traffic.pcap`
- **Baseline Generation Sequence:** `ping -c 10 127.0.0.1`
- **Forensic Verification:** An inspection verify command (`ls -lh lo_traffic.pcap`) confirms successful system capture, generating a 2.3 KB raw packet trace file containing clean ICMP echo-request and echo-reply sequences.

```
kali-linux-2025.4-virtualbox-amd64 [Running] - Oracle VirtualBox
File Machine View Input Devices Help

kali@kali: ~
Session Actions Edit View Help
command 'dcdump' from deb dicom3tools
command 'ncdump' from deb netcdf-bin
Try: sudo apt install <deb name>

(kali@kali)-[~]
$ tcpdeump --version
Command 'tcpdeump' not found, did you mean:
command 'tcpdump' from deb tcpdump
Try: sudo apt install <deb name>

(kali@kali)-[~]
$ tcpdump --version
tcpdump version 4.99.5
libpcap version 1.10.5 (with TPACKET_V3)
OpenSSL 3.5.4 30 Sep 2025
64-bit build, 64-bit time_t

(kali@kali)-[~]
$ sudo tcpdump -i lo -w lo_traffic.pcap
[sudo] password for kali:
tcpdump: listening on lo, link-type EN10MB (Ethernet), snapshot length 262144
^C20 packets captured
40 packets received by filter
0 packets dropped by kernel

(kali@kali)-[~]
$ ls -lh lo_traffic.pcap
-rw-r--r-- 1 tcpdump tcpdump 2.3K May 20 14:46 lo_traffic.pcap

(kali@kali)-[~]
$ ls
Desktop Downloads Music Public Videos
Documents lo_traffic.pcap Pictures Templates

(kali@kali)-[~]
$ _
```

2.2 Availability Disruption Attack Simulation (Day 2)

A high-frequency Denial of Service (DoS) attack was launched against local target systems to map behavior changes when a service is overwhelmed.

- **Simulation Command Syntax:** `sudo hping3 -S -p 80 --flood 127.0.0.1`
- **Observed Metrics:** The simulation utility pushed over 2.8×10^6 packets through the interface socket in seconds.
- **Forensic Analysis:** Because the data was collected using an out-of-band approach, the system was flooded with unacknowledged SYN flags. This quickly fills up the server's handshake buffer queue with unresolved, half-open connections. In a real-world production environment, this completely drains system memory resources and locks out legitimate users.

2.3 Transitioning to Inline Mitigation (Day 3)

To simulate the immediate drop-action capabilities of an operational IPS engine, a strict firewall filtering rule was injected directly into the kernel's active network stack lines.

- **Inline Prevention Command:** `sudo iptables -A INPUT -p tcp --dport 80 -j DROP`
- **Observed Behavior Response:** When the hping3 flood was re-executed, the incoming TCP requests were matched by the netfilter engine and discarded instantly at the lowest driver layer. The destination port sockets stayed clean and completely protected from connection backlog exhaustion.

```
kali-linux-2025.4-virtualbox-amd64 [Running] - Oracle VirtualBox
File Machine View Input Devices Help

kali@kali: ~
Session Actions Edit View Help

(kali@kali)-[~]
$ sudo hping3 -S -p 80 --flood 127.0.0.1
[sudo] password for kali:
HPING 127.0.0.1 (lo 127.0.0.1): S set, 40 headers + 0 data bytes
hping in flood mode, no replies will be shown
^C bit build, 64-bit time_t
— 127.0.0.1 hping statistic —
4205778 packets transmitted, 0 packets received, 100% packet loss
round-trip min/avg/max = 0.0/0.0/0.0 msap
[sudo] password for kali:
(kali@kali)-[~] on lo, link-type EN10MB (Ethernet), snapshot length 262144
$ _packets captured
40 packets received by filter
0 packets dropped by kernel

(kali@kali)-[~]
$ ls -l /var/log/lo_traffic.pcap
-rw-r--r-- 1 tcpdump tcpdump 2.3K May 20 14:46 /var/log/lo_traffic.pcap

(kali@kali)-[~]
$ ls
Desktop Downloads Music Public Videos
Documents lo_traffic.pcap Pictures Templates

(kali@kali)-[~]
$ sudo tcpdump -i lo -w dos_attack.pcap
tcpdump: listening on lo, link-type EN10MB (Ethernet), snapshot length 262144
^C9445752 packets captured
18914500 packets received by filter
22996 packets dropped by kernel

(kali@kali)-[~]
$ sudo iptables -A INPUT -p tcp --dport 80 -j DROP

(kali@kali)-[~]
$
```

Task 03: Custom Threat Signature Engineering

Custom Suricata-formatted rules sets were deployed inside the local configuration tree (./custom_rules/local.rules) using the core structural formatting syntax:

```
$$\text{action} \quad \text{protocol} \quad \text{source\_ip} \quad \text{source\_port} \quad \text{dest\_ip} \quad \text{dest\_port} \quad (\text{options})$$
```

Plaintext

Day 4: Custom Baseline ICMP Protocol Ingestion Hook

```
alert icmp any any -> any any (msg:"ICMP Echo Request Detected"; itype:8; sid:1000002; rev:1;)
```

Day 4: Custom Web Service Visibility Verification Hook

```
alert tcp any any -> any 80 (msg:"HTTP Traffic Detected"; sid:1000003; rev:1;)
```

Day 5: Custom Dynamic Rate Limiting Threshold Attack Mitigation Hook

```
alert tcp any any -> any 80 (msg:"SYN Flood Detected"; flags:S; threshold:type both, track by_src, count 20, seconds 1; sid:1000004; rev:1;)
```

Day 6: Custom Layer 2 Man-In-The-Middle Positioning Hook

```
alert arp any any -> any any (msg:"ARP Spoofing Detected"; sid:1000005; rev:1;)
```

3.1 Signature Advanced Parameters Structural Deep-Dive

- **itype:8:** Matches against ICMP Type 8 codes, allowing the analyzer to capture incoming echo requests (pings) while ignoring unneeded background diagnostic codes.
- **flags:S:** Forces the pattern engine to look for TCP packets with only the synchronization initialization bit set in the header.
- **threshold:type both, track by_src, count 20, seconds 1:** Implements a high-precision threshold matrix. The rule will only trigger an alert if a single unique source address makes 20 or more concurrent connection attempts inside a 1-second window. This isolates malicious automated tools while avoiding false alarms from regular user web requests.


```
kali-linux-2025.4-virtualbox-amd64 [Running] - Oracle VirtualBox
File Machine View Input Devices Help

kali@kali: ~
Session Actions Edit View Help

(kali@kali)-[~]
$ ping -c 10 127.0.0.1
PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.195 ms
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.027 ms
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.025 ms
64 bytes from 127.0.0.1: icmp_seq=4 ttl=64 time=0.027 ms
64 bytes from 127.0.0.1: icmp_seq=5 ttl=64 time=0.025 ms
64 bytes from 127.0.0.1: icmp_seq=6 ttl=64 time=0.027 ms
64 bytes from 127.0.0.1: icmp_seq=7 ttl=64 time=0.025 ms
64 bytes from 127.0.0.1: icmp_seq=8 ttl=64 time=0.026 ms
64 bytes from 127.0.0.1: icmp_seq=9 ttl=64 time=0.026 ms
64 bytes from 127.0.0.1: icmp_seq=10 ttl=64 time=0.027 ms
0 packets dropped by kernel
— 127.0.0.1 ping statistics —
10 packets transmitted, 10 received, 0% packet loss, time 9213ms
rtt min/avg/max/mdev = 0.025/0.043/0.195/0.050 ms
-rw-r--r-- 1 tcpdump tcpdump 2.3K May 20 14:46 lo_traffic.pcap

(kali@kali)-[~]
$ _
Desktop Downloads Music Public Videos
Documents lo_traffic.pcap Pictures Templates

(kali@kali)-[~]
$ sudo tcpdump -i lo -w dos_attack.pcap
tcpdump: listening on lo, link-type EN10MB (Ethernet), snapshot length 262144
^C9445752 packets captured
18914500 packets received by filter
22996 packets dropped by kernel

(kali@kali)-[~]
$ sudo iptables -A INPUT -p tcp --port 80 -- DROP

(kali@kali)-[~]
$
```

Task 04: Incident Telemetry Processing & Output

4.1 Production Logging Management Engines

When deployed across an enterprise network environment, Suricata processes matched events into two clear logging formats:

1. **/var/log/suricata/fast.log:** A single-line, lightweight text log designed for rapid system monitoring. It outputs basic information like timestamp, Message String (msg), Signature ID (sid), and IP/Port endpoints.
2. **/var/log/suricata/eve.json:** An advanced JavaScript Object Notation (JSON) logging database. This format tracks deep transaction context, layer-7 protocol breakdowns, metadata states, and packet dump structures. It is ideal for ingestion into Security Information and Event Management (SIEM) systems for complex correlation mapping.

Task 05: Defending Layer 2 Protocol Mechanics

5.1 ARP Spoofing Vector and Structural Risk Exposure (Day 6)

The Address Resolution Protocol (ARP) is an unauthenticated, stateless layer-2 infrastructure protocol designed to translate IP layer mappings into hardware MAC addresses. Because it has no built-in verification rules, an attacker can use tools like arpspoof (sudo arpspoof -i eth0 -t <victim-ip> <gateway-ip>) to send unauthorized MAC address updates to local switches.

This forces network equipment to re-route internal enterprise traffic directly through the attacker's system, allowing them to capture unencrypted data streams (like plain-text HTTP transactions on port 80) or execute Man-in-the-Middle (MitM) session high-jacking.

5.2 Enterprise Detection Engineering Strategy

While changing rule actions from alert to drop can automatically mitigate floods, deploying automated inline drop rules across modern enterprise architectures can be risky. If rules are configured too tightly or trigger on false positives, they can block critical business channels and cause unexpected outages.

To keep operations resilient, security teams should follow a layered protection strategy:

- **Enforce Security Boundaries:** Move web and application systems away from unencrypted HTTP (Port 80) and require full encryption using HTTPS (Port 443), TLS 1.3, and robust VPN tunnels.
- **Deploy Hardened Infrastructure Controls:** Turn on **DHCP Snooping** and **Dynamic ARP Inspection (DAI)** on core enterprise switch devices to block unauthorized MAC address updates at the hardware level.
- **Correlate Traffic Behavior:** Pair rule-based engines with behavior tracking analytics to catch anomalous traffic speeds and sudden configuration modifications before they can affect system availability.

Shakera Karolia | CSI-B1-516 | Page 11

Cyberster Purple Team Internship | Batch 1 Month 2

#.

Submitted by: Shakera Karolia Roll No: CSI-B1-516 Cyberster Purple Team Internship — Batch 1 Week 5 Report 20 May 2026

Shakera Karolia | CSI-B1-516 | Page 13